

von

Praxis der Softwareentwicklung

Dokumentation der Hausarbeit

Gruppe C7

Lars Nicht, Jannick Gottschalk, Daria Stolarczyk, Viktoria Melnyk

Inhaltsverzeichnis

1. Einleitung (Viktoria)	3
2. Softwareübersicht (Viktoria).....	3
3. Installationsanleitung (Lars)	4
3.1 Voraussetzungen: Notwendige Software zur Verwendung.	4
3.2 Installation: Schritt-für-Schritt-Anleitung zum Import des Projekts in IntelliJ.	5
3.3 Ausführung: Schritt-für-Schritt-Anleitung zur Ausführung der Software.....	5
4. Benutzerhandbuch (Jannick).....	5
4.1 Benutzeroberfläche	5
4.2 Anwendungsszenarien	5
4.3 Mögliche Fehler	6
5. Entwicklerdokumentation (Daria, Jannick, Lars)	6
5.1 Ordner Struktur	6
5.2 Logische Komponenten	7
5.2.1 Core Logic.....	7
5.2.2 Experiment, Konfigurationen, Logging	9
6. Testdokumentation (Lars)	10
6.1 Teststrategie: Überblick über die Testarten	10
6.2 BaseExperimentTests & ExperimentTests: Besonderer Fokus auf die Gestaltung der BaseExperimentTests, sowie der Dynamik der einzelnen Experiment Tests	10
7. Wartung und Erweiterung (Viktoria).....	11
8. Anhang (Viktoria).....	12
Glossar:	12
Quellen:	13
Technische Hilfsmittel:	13

1. Einleitung (Viktoria)

Die folgende Dokumentation dient zur Beschreibung der Hausarbeit im Rahmen des Moduls „Praxis der Softwareentwicklung“.

Das Ziel der Hausarbeit ist es einen 2-dimensionalen zellulären Automaten zu programmieren, welcher anhand der Regeln „Game of Life“ und „Parity“ den weiteren Lebensverlauf der Zellen innerhalb eines Grids berechnet. Mithilfe einer festgelegten Startkonfiguration und dem gewählten Nachbarschaftsmodell wird der Zustand jeder Zelle im Grid iterativ über aufeinanderfolgende Berechnungsschritte ermittelt, bis eine vorgegebene Abbruchbedingung erreicht ist.

Die Berechnungsschritte können innerhalb der Konsole, einer Datei oder in Kombination festgehalten werden, um die Berechnung auch im Nachhinein nachvollziehen zu können.

2. Softwareübersicht (Viktoria)

Die Architektur der Software folgt einem modularen Ansatz, der durch den Einsatz des Strategy Patterns unterstützt wird. Um den Aufbau der Software zu erläutern, werden die verschiedenen Komponenten, deren Verantwortlichkeiten und die Beziehungen verschiedener Komponenten untereinander mithilfe eines UML-Diagramms dargestellt.

Das UML-Klassendiagramm ist in den Anhängen zu finden.

Durch die Architektur wird das Ziel der einfachen Wartbar-, Erweiter-, und Ersetzbarkeit verfolgt.

Ein zentrales Merkmal dieser Architektur ist die ausgegliederte Logik in Strategie Klassen. Dadurch können verschiedene Berechnungsstrategien oder Nachbarschaftsmodelle angepasst, ersetzt oder neu hinzugefügt werden, ohne dass Änderungen an anderen Teilen der Software erforderlich sind. Dies stützt die Erweiterbarkeit der Software.

Die Hauptkomponenten der Architektur sind die Zelle, das Grid, die Strategy, das Experiment und der Logger.

Das *model-Package* bildet die Grundlage der zellulären Automaten-Software und enthält die zentrale *Cell-Klasse*, die eine einzelne Zelle im Grid repräsentiert. Die Zelle bildet das Basiselement, da sie den Lebenszustand und die Position jeder Zelle im Grid definiert. Diese Klasse definiert die Position (*row*, *column*) und den Zustand (*isAlive*) einer Zelle und bietet Methoden zum Zugriff und zur Verwaltung dieser Attribute.

Das Grid bildet die zentrale Schnittstelle der Core Logik im Automaten. Es greift auf die Interfaces oder Klassen der Zelle und Strategien zu. Die Hauptaufgabe dieser Komponente ist die Berechnung der nächsten Generation des Grids, welche abgebildet wird durch die Änderungen der Zellzustände in Kombination mit ihren Nachbarn und Überlebensregeln. Im *grid-Package* werden innerhalb des *IGrid Interfaces* die grundlegenden Methoden für alle Grid-Typen festgelegt, wie die Berechnung der nächsten Generation (*calculateNextGeneration*), der Zugriff auf Zellen (*getCellByCoordinates*), und die Stabilitätsprüfung (*isStable*).

Die Grid-Implementierungen finden über die Klassen *ClassicGrid* und *SetGrid* statt.

Die Klasse *ClassicGrid* nutzt ein zweidimensionales Array zur expliziten Speicherung aller Zellen. Die Klasse *SetGrid* speichert hingegen nur lebende Zellen in einem *HashSet*, was für speichereffizientere.

Das *GridType Enum* listet die Grid-Typen auf und erleichtert die Auswahl des gewünschten Typs. Die *GridFactory* erstellt Grid-Instanzen basierend auf der Benutzereingabe.

Die Strategy-Komponenten ermöglichen den Einsatz verschiedener Logiken zur Berechnung des Zellzustands und der Zellenachbarn. Das Strategy Pattern sorgt dafür, dass die Berechnungslogik abgekapselt und implementierungsunabhängig ausgewählt werden kann.

Die Experiment-Komponente steuert die Durchführung der Simulationen. Jedes Experiment enthält eine spezifische Startkonfiguration mit der das Grid initialisiert wird, setzt die vom User ausgewählten Strategien, lässt das Experiment laufen und prüft die Abbruchbedingungen.

Der Logger ist dafür verantwortlich, die Simulationsergebnisse zu protokollieren und für die Nachvollziehbarkeit der Berechnungsschritte zu dokumentieren. Es gibt verschiedene Logger-Implementierungen, wie etwa die Ausgabe in die Konsole, die Speicherung in einer Datei oder eine Kombination aus beiden.

Grundsätzlich wurden Interfaces als Schnittstellen zur Verfügung gestellt, wenn mehrere Implementierungslogiken für eine Komponente realisiert werden sollten.

Bei Gemeinsamkeiten im Code wurde noch eine weitere Abstraktionsebene eingeführt über eine abstrakte oder eine Super-Klasse.

Um eine Instanz der Schnittstellen zu erstellen, wurde das Factory Pattern eingesetzt, welches durch Enums und Valuezuweisung der Enums die gewünschte Instanz erstellt.

3. Installationsanleitung (Lars)

Für eine detailliertere und besser übersichtliche Installations- und Nutzungsanleitung nutzen Sie bitte die [README.md](#) Datei in GitLab.

3.1 Voraussetzungen: Notwendige Software zur Verwendung.

Um das Projekt zu starten, müssen Sie folgende Software installiert haben. Als erstes ist **Java Development Kit (JDK) 21** oder eine neuere Version erforderlich, da dies die Basis für den Code des Projekts bildet. Außerdem wird die Verwendung von **IntelliJ IDEA** als Entwicklungsumgebung empfohlen, da wir sie für die Entwicklung für dieses Projekts genutzt haben. Die dritte Voraussetzung ist **Apache Maven** lokal zu installieren, sie wird nur zum Ausführen der “**mvn**”-Commands in der README.md benötigt und ist optional. Das Abhängigkeitsmanagement von Maven wird

automatisch beim Importieren des Projekts genutzt. Falls Sie diese Programme noch nicht installiert haben, sollten Sie diese herunterladen und einrichten, bevor Sie zum nächsten Schritt gehen.

3.2 Installation: Schritt-für-Schritt-Anleitung zum Import des Projekts in IntelliJ.

Öffnen Sie zunächst IntelliJ IDEA und wählen Sie im Menü den Punkt **'File' -> 'New' -> 'Project from Version Control'** oder **'Get from VCS'**. Anschließend wählen Sie Git als Versioning und geben die URL des Repositorys ein: **'https://gitlab2.nordakademie.de/LarsNicht-I22/hausarbeit_i143_2024_i22c_c7.git'**. Sobald Sie die URL eingegeben haben, klicken Sie auf **'Clone'** und geben Sie Ihre Nutzerdaten für das NAK-GitLab ein. Anschließend importiert IntelliJ IDEA das Projekt automatisch und lädt über Maven alle erforderlichen Abhängigkeiten herunter. Danach sollte das Projekt erfolgreich in Ihrer Entwicklungsumgebung eingerichtet sein.

3.3 Ausführung: Schritt-für-Schritt-Anleitung zur Ausführung der Software.

Nachdem das Projekt erfolgreich importiert wurde, können Sie die Anwendung starten. Sie können das Programm über das GUI von IntelliJ IDEA starten oder per Maven-Command (siehe README.md). Navigieren Sie dafür in IntelliJ zum Verzeichnis **'src/main/java/de/nordakademie/zellulaere_automaten'** und klicken Sie mit der rechten Maustaste auf die Klasse **'ZellulaereAutomatenApplication'**. Im Kontextmenü wählen Sie die Option **Run 'ZellulaereAutomatenApplication.main()'**. Damit wird die Anwendung direkt aus der IDE heraus gestartet. Für eine Beschreibung zur Erzeugung und Nutzung einer .jar Datei des Projekts, lesen Sie bitte die README.md.

4. Benutzerhandbuch (Jannick)

4.1 Benutzeroberfläche

Sobald Sie das Programm gestartet haben, startet zunächst die Spring-Application und zeigt kurze Informationen zum System in der Konsole an. Anschließend finden eine Reihe von Nutzerabfragen über die Konsole zu den Modellarten, den Experimenten, den Startkonfigurationen sowie dem zu benutzenden Log-Typ statt.

Zunächst jedoch steht Ihnen die Möglichkeit offen, alle Verfügbaren Experimente auszuführen mit dem Befehl **'all'** auszuführen. Bei jeder Auswahl werden die zur Verfügung stehenden Optionen angezeigt und können mit der zugeordneten Zahl bestätigt werden

4.2 Anwendungsszenarien

Nehmen wir uns nun als Beispiel die Ausführung des Experiments **'GameOfLifeExperiment1Structure1'** vor.

Nach Ausführung, erfragt Sie das Programm, welches Modell Sie ausführen wollen. Da wir in diesem Beispiel ein GameOfLife-Modell anstreben, geben wir eine 1 für ‘GameOfLife mit Moore’ in die Konsole ein und bestätigen unsere Eingabe mit der Enter-Taste.

Anschließend fragt die Applikation nach dem gewollten Experiment. Die Experimente unterscheiden sich in der Größe des Grids und in ihrem Startmuster. Diese sind in der Aufgabenstellung der Hausarbeit definiert. Wir wählen in unserem Beispiel die 1 für Startkonfiguration 1 beziehungsweise Experiment 1.

Nachdem wir nun erfolgreich unser Modell sowie unser Experiment ausgewählt haben, fragt die Applikation nach der Struktur beziehungsweise der Art des Grids. Da wir die erste Struktur wählen wollen, geben wir eine 1 für ‘ClassicGrid’ ein und bestätigen diese ebenfalls.

Zu guter Letzt wählen wir die Art des Logvorgangs. Diese besteht aus 4 Möglichkeiten, die Sie nach Bedarf wählen können. Dazu zählen 0 für NoLog (kein Log wird geschrieben), 1 für LogFile (eine Datei mit der Endung .log wird geschrieben), 2 für LogConsole (das Log wird in der Konsole ausgegeben) und 3 für LogConsoleAndFile (in dem das Log in einer Datei sowie in der Konsole ausgegeben wird).

Nach Bestätigung der letzten Auswahlmöglichkeit wird das Programm gestartet. Bei der Wahl einer Aufgabe in eine Datei, wird die Datei nach dem Namen des gewählten Experiments und der Endung .log benannt (in unserem Fall: ‘GameOfLifeExperiment1Structure1.log’) und in der obersten Dateiebene des Projekts gespeichert.

Falls Sie jedoch das Projekt über eine Jar ausführen, werden die Log-Dateien in dem Ordner gespeichert, in dem sich diese befindet. Damit ist der Prozess abgeschlossen.

Wenn Sie alle Experimente ausführen wollen, können Sie bei der ersten Abfrage ‘all’ eingeben. Nach der anschließenden Wahl des Logvorgangs wird der Prozess gestartet.

4.3 Mögliche Fehler

Durch die eingebaute Validierung der Benutzereingaben können Fehler nur in sehr geringem Umfang und auch nur in diesem Umfeld auftauchen. Sollten Sie bei einer der in 4.2 beschriebenen Benutzereingaben, eine Falsche Eingabe tätigen, wird Ihnen erneut die Möglichkeit einer Eingabe gewährt.

5. Entwicklerdokumentation (Daria, Jannick, Lars)

Im folgenden Abschnitt wird das Projekt technisch dokumentiert.

5.1 Ordner Struktur

Die Ordnerstruktur folgt im Sourceordner dem “main”, also dem Sourcecode, und dem “test”, wo sich alle Testdateien befinden.

Unter dem Path: “java/de.nordakademie.zellulaere_automaten” finden sich die logischen Komponenten zusammengefasst in ihren Ordnern. Hier sieht man, dass die bereits oben erwähnten Komponenten:

- Experiment
- Grid
- Logger
- Model
- Strategy
- Die Startklasse mit dem Einstiegspunkt “ZellulaereAutomatenApplication”
- Und den Inputhandler, der alle benötigten Variablen für den Aufbau des Experiments vom User abfragt.

Die Testordnerstruktur folgt analog.

5.2 Logische Komponenten

5.2.1 Core Logic

Die “Core Logik” besteht aus drei Komponenten, welche im folgendem vorgestellt werden und als Basiselemente gelten:

- Cell (Im Package Model)
- Die Strategy Klassen
- Grid

Die “Cell”

ist das Objekt, welches die Zelle beschreiben soll. Zur eindeutigen Identifikation speichert die Zelle die Koordinaten im Grid: “row” und “column”. Es gibt noch ein weiteres Attribut, den “isAlive” Status. In der Zelle werden Methoden “overridden” von “Object” dazu gehören: “toString”, “equals”, “hashCode”. Diese Methode ermöglichen Erleichterungen für das Logging und das Handling der Objekte in Datenstrukturen der Java Collection.

Die “Strategys”

sind in zwei Komponenten aufgeteilt:

1. Neighbors: Auswahl der Nachbarn für eine Zelle
2. StateCalculation: Berechnung des Zellenzustands mit dem gegebenen Nachbarn

Diese Logiken sollten erweiterbar als auch austauschbar sein, somit sind sie über die Verwendung des “Strategy Patterns” in eigene Klassen ausgelagert. Der gewählte Aufbau ist ein Interface, welches die zu implementierenden Methoden festlegt.

In der “Neighbor Strategy” ist die Moore Logik eine Erweiterung der von-Neumann Logik, somit benutzen beide die Logik der Nachbarn von oben, unten, links, rechts einer Zelle. Aufgrund dessen wurde diese Logik in eine abstrakte Klasse ausgelagert.

Da der Aufruf dieser Klassen möglichst lose gekoppelt sein soll an die anderen logischen Komponenten, wird die Erstellung einer Instanz über **das “Factory Pattern”** ausgeführt.

Es wird ein “Enum” angelegt mit den vorhandenen Typen an vorhandenen Implementierungen. Zusätzlich wird diesen ein Zahlenwert zugewiesen und es sind zwei “Getter” vorhanden, um jeweils von dem Typ auf die Zahl und andersherum abrufen zu können.

Anmerkung zur Architektur

Durch diese Architektur kann anschließend ein geparster Userinput in die Factory gegeben werden, um die korrekte Instanz zu erzeugen.

Diese Logik ist in fast jeder logischen Komponente wiederzufinden, demnach wird im Folgenden diese Architektur nicht wiederholt erklärt und gilt analog für alle weiteren Komponenten. Während dieser Ansatz einen größeren Klassenumfang mit sich bringt, erfüllt der Code das Prinzip der losen Kopplung und fördert durch viele, aber schlankere Klassen schnelle Fehlererkennung als auch klare Verantwortungsbereiche.

Das Grid

ist als komplexere Komponente, die erste Zusammenlaufstelle für die “Core Logic”.

Es vereint die Zelle, Nachbarlogik und Berechnungsregeln, um den Hauptalgorithmus, die Berechnung der nächsten Generation der Zellen auf dem Grid, zu realisieren.

Dies wurde in zwei Datenstrukturen umgesetzt:

1. ClassicGrid: zweidimensionales Array
2. SetGrid: Hashset

Von beiden Klassen wird ebenfalls “toString” implementiert mit Benutzung des “toStrings” der Zellen Klasse. Dies ermöglicht dem Logger jedes Grid unabhängig von der unterliegenden Datenstruktur darzustellen.

Das Classic Grid

verfolgt den naiven Ansatz eines zweidimensionalen Arrays, welches initialisiert wird mit der “Cell” und der gegebenen Startkonfiguration. Über eine doppelte “For-Loop” werden für jede Zelle die Nachbarn abgerufen und eine Berechnung des Lebens Status ausgeführt.

Für den Stabilitätscheck wird vor jeder Ausführung eine Kopie des vorherigen Grids gespeichert, bevor die Änderungen der Berechnung im aktuellen Grid abgespeichert werden.

Das Set Grid

Speichert nicht das gesamte Grid, sondern nur eine Liste, der aktuell im Kontext aktiven Zellen. Genau wie im ClassicGrid gibt es eine Kopie dieser Liste zur Überprüfung der Stabilität des Grids. Hierbei wird auf den “Override” der Zelle von “equals” zurückgegriffen und kann somit elegant verglichen werden. Die Berechnung des neuen Zustandes wird über Sets realisiert, da hier auch doppelte Einträge nicht aufgenommen werden. Aus dem aktiven Set werden die zu überprüfenden Zellen ermittelt, welche in einem Hashset abgespeichert werden. Dies wird vor allem für Performance Gründe so gemacht. Somit wird sichergestellt, dass keine Zellnachbarschaft doppelt berechnet wird. Um dies umzusetzen, wird eine Map Funktion “computeIfAbsent” benutzt, welche die Berechnung nur macht, wenn die Zelle nicht bereits in der Hashmap vorhanden ist.

Anschließend wird ein neues Set “nextGeneration” befüllt und am Ende dem “activeCell” Set zugewiesen.

5.2.2 Experiment, Konfigurationen, Logging

Die Logger Komponente

Der Logger kann in verschiedenen Modi ausgeführt werden, hierzu wird Interface die gegebene Logik definiert und in den jeweiligen Klassen implementiert:

- LogConsole
- LogFile
- LogConsoleAndFile
- NoLog

die für die dementsprechende Logger-Funktionalität stehen.

LogConsoleAndFile erschafft lediglich die beiden Klassen LogFile und LogConsole um beide Funktionalitäten abzudecken und Erweiterbarkeit durch eine Erwähnung im LoggerTypes-Enum zu ermöglichen. NoLog schreibt kein Log, sondern schreibt eine Mitteilung in die Konsole, die dem User verständlich macht, dass kein Log geschrieben wird. LogConsole gibt das Log inklusive Header und Body in die Konsole aus. Die inkludiert ein Andruck des aktuellen Experiments vor Andruck der initialen Experiment-Iteration.

LogFile druckt die Iterationen eines Experiments in eine Datei die wie das entsprechende Experiment benannt werden soll sowie der Endung '.log'

BaseExperiment - Die Grundlage für die Experimente

Die BaseExperiment-Klasse ist die Grundlage für alle Experimente (GameOfLifeExperiment1Strucure1 etc.) und implementiert das IExperiment-Interface. Sie stellt mehrere Methoden für die Experimente bereit, welche durch diese überschrieben werden. Die Methode 'initializeGrid()' wird in den abgeleiteten Klassen überschrieben und initialisiert je nach Experiment entsprechend das Grid mit einer Startkonfiguration. 'runExperiment()' führt die Experimente nach entsprechendem Aufrufen aus und nimmt LogType als Parameter an. In der Methode befinden sich Aufrufe an den Logger (z.B. logEndMessage) und das Grid (calculateNextGeneration) um die korrekte Berechnung der Experimente zu gewährleisten, sowie sich der Check nach den Abbruchbedingungen innerhalb dieser Methode befindet.

Die Experimente - GameOfLife und Parity

Die Experimente in den Packages 'gameOfLife' und 'parity' erweitern die BaseExperiment-Klasse und implementieren das IExperiment-Interface. Diese Klassen stellen vordefinierte Experimente mit Konfigurationen in Bezug auf Grid-Größe, Grid-Type Neighbourhood-Model und Initialisierungspattern dar. Der Konstruktor der jeweiligen Klasse initialisiert eine neue Instanz dieses Experiments. Er ruft den Konstruktor seiner Oberklasse (BaseExperiment) mit den jeweiligen Konfigurationen auf. Die Experiment-Klassen überschreiben die Methode initializeGrid() von BaseExperiment und erstellen somit die richtige Startkonfiguration. Alle lebendigen Zellen werden der Startkonfiguration hinzugefügt, welche dann in runExperiment() genutzt wird.

Hierbei ist zu erwähnen, dass das Hardcoden der Experimente von unserer Seite nicht gewünscht war. Es ist ersichtlich, dass die Software so aufgebaut ist, dass anhand des Userinputs das Experiment dynamisch hätte erstellt werden können.

5.2.3 Einstiegspunkt und Inputhandler

Der Inputhandler wird über die main-Methode aufgerufen und setzt das Einlesen des Userinputs sowie die Validierung des Inputs um. Zudem beinhaltet die Klasse die Funktion `executeAllExperiments()`, die alle Experimente durchlaufen lässt. Außerdem verfügt die Methode `executeAllExperiments()` über eine "Effizienzanalyse", da sie mit Hilfe von `java.time` die Zeit zur Durchführung der Experimente aufführt und somit Vergleichswerte aufzeigt.

6. Testdokumentation (Lars)

6.1 Teststrategie: Überblick über die Testarten

Für dieses Projekt wurde test-first getestet, was bedeutet, dass die meisten Tests vor der Implementierung der eigentlichen Funktionalitäten geschrieben wurden. Einige Tests wurden im Nachhinein noch weiter angepasst und verbessert. Als Testframeworks haben wir JUnit und Mockito genutzt. JUnit dient als Basis für die Teststruktur, während Mockito gezielt für das Mocking von Abhängigkeiten genutzt wurde. Mockito wurde dort genutzt, wo wir die Nutzung als sinnvoll erachtet haben. Insgesamt folgt die Teststrategie des Projekts dem Prinzip der Modularität und Wiederverwendbarkeit (auch wenn wir durch die Vorgaben ein bisschen ausgebremst wurden, siehe die Experiment Klassen).

6.2 BaseExperimentTests & ExperimentTests: Besonderer Fokus auf die Gestaltung der BaseExperimentTests, sowie der Dynamik der einzelnen Experiment Tests

Bei den **BaseExperimentTests** liegt der Fokus auf der Gestaltung der Tests auf Modularität. Die Tests bilden die Basis, auf der alle spezifischen Experiment-Tests aufbauen, und stellen sicher, dass zentrale Funktionen und Strukturen einheitlich und zuverlässig funktionieren. Mithilfe parametrisierter Tests und deren Konfigurationen wird gewährleistet, dass alle Grid-Typen, sowie alle Modelle und Algorithmen korrekt arbeiten und die Hauptfunktionen der Experimente unabhängig von spezifischen Parametern stabil bleiben. Eine zentrale Rolle spielt hierbei die `gridProvider()`-Methode. Durch den Aufbau der `gridProvider()`-Methode können dieselben Tests flexibel auf die Grid-Typen und die verschiedenen Algorithmen angewendet werden. Dies stellt eine flexible Testabdeckung sicher und macht die Tests theoretisch erweiterbar für zusätzliche Grid-Typen.

Die **Experiment Tests** bauen auf dieser Grundlage auf und erweitern sie. Für jedes Experiment existiert eine eigene Testklasse, die dynamisch an die Grid-Größe angepasst wird, solange das

Seitenverhältnis des Grids gleichbleibt. Dabei gibt es jedoch bestimmte Grenzen: Beispielsweise muss bei GameOfLifeExperiment1 die Grid-Größe größer sein als das Startmuster, um korrekt zu funktionieren. Nur der Test `getGridDimensions_WhenCalled_ShouldReturnCorrectDimensions` überprüft explizit die exakte Größe des Grids und würde bei einer Anpassung der Grid-Größe fehlschlagen. Zusätzlich validieren die Tests die Initialisierung des Grids der einzelnen Experimente, sowie die erste erzeugte Generation mithilfe von vordefinierten Mustern, um sicherzustellen, dass die Experimente den erwarteten Startzustand und die initiale Entwicklung korrekt umsetzen.

- **Automatisierung und Testumgebung:** Informationen zur Ausführung der Tests und zur Testumgebung.

7. Wartung und Erweiterung (Viktoria)

Die Aktualisierung der Software auf eine neue Version erfolgt nach einem klar strukturierten Prozess, um eine verlässliche und effiziente Implementierung sicherzustellen. Zu Beginn wird eine vollständige Sicherung der aktuellen Projektkonfigurationen, Daten sowie der experimentellen Startkonfigurationen erstellt, um im Falle von Problemen eine Wiederherstellung zu ermöglichen. Anschließend wird die neueste Version der Software aus dem Git-Repository heruntergeladen und installiert. Dabei müssen alle relevanten Abhängigkeiten, wie etwa Maven-Bibliotheken und die Java-Laufzeitumgebung (OpenJDK 21), überprüft und gegebenenfalls aktualisiert werden, um eine einwandfreie Kompatibilität sicherzustellen.

Nach erfolgreicher Installation werden die gesicherten Einstellungen und Konfigurationen in die neue Version übertragen, sodass die bisher definierten Parameter nahtlos übernommen werden. Dieser Schritt gewährleistet, dass die Software weiterhin wie gewohnt funktioniert, ohne dass zusätzliche manuelle Anpassungen erforderlich sind. Ein zentraler Bestandteil der Aktualisierung ist die Durchführung umfangreicher Testläufe, um sicherzustellen, dass alle bestehenden und neuen Funktionen wie gewünscht arbeiten. Hierfür werden Unit-Tests mit JUnit sowie Mock-Tests mit Mockito verwendet, um die Integrität und Funktionsfähigkeit der Module zu gewährleisten. Diese Vorgehensweise garantiert, dass die Software auch nach dem Update stabil bleibt und alle Experimente weiterhin zuverlässig durchgeführt werden können.

Die Erweiterbarkeit der Software spielt eine zentrale Rolle im Design. Dank der Verwendung von Entwurfsmustern wie dem Strategy Pattern und der modularen Struktur der Hauptkomponenten lässt sich die Software problemlos an sich verändernde Anforderungen anpassen. Neue Funktionen können durch die Integration zusätzlicher Strategien, das Erweitern der Nachbarschaftsmodelle oder das Hinzufügen weiterer Experimentvarianten einfach implementiert werden, ohne dass bestehende Systemteile verändert werden müssen. Dieser Ansatz gewährleistet eine hohe Flexibilität in der Entwicklung und ermöglicht es, auf zukünftige Anforderungen schnell und effizient zu reagieren.

Potenzielle zukünftige Anforderungen könnten die Implementierung zusätzlicher Berechnungsstrategien, die Einführung weiterer Nachbarschaftsmodelle wie dem hexagonalen

Modell, die Entwicklung einer grafischen Benutzeroberfläche (GUI) zur verbesserten Visualisierung der Simulation sowie die Erweiterung der Logger-Funktionalität um umfassendere Statistiken und Exportmöglichkeiten umfassen.

Das Fehlermanagement der Software umfasst Verfahren zur Protokollierung und Behebung von Fehlern, um eine hohe Stabilität und Verlässlichkeit sicherzustellen. Fehler werden mithilfe eines integrierten Logger-Systems erfasst, das sämtliche Ereignisse und ungewöhnlichen Zustände in der Konsole und/oder in Log-Dateien dokumentiert. Diese Protokollierung stellt sicher, dass die Fehlerquellen schnell identifiziert werden können. Die Protokolle dienen als Basis für eine systematische Analyse, wodurch die Ursachen der Fehler nachvollzogen und gezielt behoben werden können. Zur effektiven Fehlerbehebung werden auch Unit-Tests mit JUnit und Mock-Tests mit Mockito eingesetzt, um sicherzustellen, dass die Fehler dauerhaft gelöst und keine neuen Probleme verursacht werden. Diese Testmethoden helfen dabei, die Integrität des Systems nach Änderungen zu gewährleisten und ermöglichen eine nachhaltige Verbesserung der Software.

8. Anhang (Viktoria)

Glossar:

Abbruchbedingung: Eine vordefinierte Bedingung, die bei Erfüllung die iterative Berechnung beendet und somit das Experiment abschließt.

ClassicGrid: Ein zweidimensionales Array, das aus Zellen besteht, die ihren Zustand basierend auf bestimmten Regeln ändern.

CoreLogic: Die Hauptkomponenten des Projekts

Game of Life: Ein Regelsatz, der den Lebensstatus einer Zelle bestimmt. Eine Zelle bleibt bei 2 oder 3 lebenden Nachbarn am Leben und stirbt in anderen Fällen. Eine tote Zelle wird wiederbelebt, wenn sie genau 3 lebende Nachbarn hat.

Iteration: Ein Berechnungsschritt, bei dem der Zustand jeder Zelle im Gitter anhand der festgelegten Regeln und ihrer Nachbarschaft aktualisiert wird.

Logger: Eine Komponente, die die iterativen Berechnungsschritte aufzeichnet und dokumentiert.

Moore-Nachbarschaft: Ein Nachbarschaftsmodell, bei dem eine Zelle von acht Nachbarn umgeben ist: links, rechts, oben, unten, links oben, rechts oben, links unten und rechts unten.

Parity: Ein Regelsatz, bei dem der Lebensstatus einer Zelle von der Anzahl lebender Nachbarn abhängt. Eine Zelle bleibt am Leben, wenn die Anzahl der lebenden Nachbarn ungerade ist, andernfalls stirbt sie.

SetGrid: Ein HashSet mit nur aktiven Zellen

Stabilität: Ein Zustand, in dem die Konfiguration des Grids unverändert bleibt und keine weiteren Änderungen an den Zuständen der Zellen auftreten.

Strategy Pattern: Ein Entwurfsmuster, das einen dynamischen Austausch ähnlicher Algorithmen ermöglicht, um Flexibilität im Verhalten des Programms zu gewährleisten.

Von-Neumann-Nachbarschaft: Ein Nachbarschaftsmodell, bei dem eine Zelle mit vier Nachbarn interagiert: oben, unten, links und rechts.

Zellulärer Automat: Ein Modell zur Simulation dynamischer Systeme, bei dem Zellen in einem Grid nach bestimmten Regeln miteinander interagieren.

Quellen:

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (2015). Design patterns (mitp professional): Entwurfsmuster als Elemente wiederverwendbarer objektorientierter Software. mitp.

Technische Hilfsmittel:

Bei der Realisierung des Projekts wurden folgende Hilfsmittel genutzt:

Spring Initializr: zur Einrichtung des Projekt Setups

Git: als Versionskontrollsystem

Maven: für das Build Management und Verwaltung von Abhängigkeiten

IntelliJ IDEA: als Entwicklungsumgebung

JDK 21: als Laufzeitumgebung für Java

Maven Repository: für Erweiterungen und Bibliotheken

JUnit (Extension): für die Implementierung und Durchführung von Tests

Mockito (Extension): als Mocking-Framework für das isolierte Testen von Komponenten

Chat GPT: als Entwicklungsassistent und Formulierungshilfe (z.B. Formatierung der README.md, JavaDoc)

Copilot: als Entwicklungsassistent (z.B. JavaDoc)

DeepL: als Übersetzungs-Tool

Selbstständigkeitserklärung

Mit unserer Unterschrift versichern wir, dass wir die hier vorliegende Arbeit selbständig, ohne fremde Hilfe und nur mit den angegebenen Hilfsmitteln verfasst haben und unsere Angaben zu den verwendeten Quellen der Wahrheit entsprechen und vollständig sind. Alle Quellen, aus denen wir wörtlich oder sinngemäß übernommen haben, haben wir als solche gekennzeichnet. Wir haben zur Kenntnis genommen, dass zuwiderlaufendes Verhalten als Täuschungsversuch gewertet wird und zu den in der geltenden Prüfungsverfahrensordnung genannten Konsequenzen führen wird.

Lars Nicht

Lars Nicht

Jannick Gottschalk

Jannick Gottschalk

Daria Stolarczyk

Daria Stolarczyk

Viktoria Melnyk

Viktoria Melnyk

